```
$ rspec                    ruby-2.4.1 security_01 0322db3 ×
Running via Spring preloader in process 51973
FF  HTML screenshot: <filename>
F  HTML screenshot: <filename>
F  HTML screenshot: <filename>
F  HTML screenshot:
<filename>
.....................................................
```

A lot of other tests fail.

When a change in your code breaks multiple tests, that's often a good time to revisit your testing strategy. It's common for new data or security constraints to break tests that are unaware of new requirements. But multiple tests breaking can be a sign that your tests are too entangled with the internals of classes that are not directly under test.

Prescription 31	When a single change in your code breaks multiple tests, consider the idea that your testing strategy is flawed.

The specific test failures are all due to code attempting to hit the site without having a login. It might mean that some of the testing you're doing at the controller level would be better done at a unit-test level against an action object. In your case, it also means you have some spurious tests floating around from Chapter 7, *Using Test Doubles as Mocks and Stubs*, on page 129.

You need to simulate a logged-in user for each failing test. There are three different failing test areas: the request specs and the two other system specs you have for adding projects and adding tasks.

You need to create a fake user that can fake logging in. There are a couple of options. You could use fixtures to create a sample user or you could use factory_bot. For the tests you've previously written, you don't need to know anything about the user; you just need to log in. Here's a simple factory_bot user factory:

security/01/spec/factories/users.rb

```
FactoryBot.define do
  factory :user do
    sequence(:email) { |n| "user_#{n}@example.com" }
    password("password")
  end
end
```

In this case you're taking advantage of factory_bot's ability to use attributes that are attributes of the class but aren't directly database columns. Devise